SIFT: A Self-Verifying Text-to-SQL Agent for Messy Relational Databases

Project Synopsis

Hackathon Track	AI-02 — Data-Analyst Agent on Messy Databases
Theme	Building Intelligent, Trustworthy and Sustainable Systems for the Future
Team	AI-02
Benchmark	BIRD (BIg Bench for Large-scale Database Grounded Text-to-SQL Evaluation)
Primary Metric	Execution Accuracy (EX) on the official dev split

1. Abstract

Business users increasingly demand natural-language interfaces to structured data. Existing text-to-SQL systems perform well on clean, well-documented databases, but fail silently when confronted with real-world conditions: cryptic column names, dirty values such as "N/A" strings and mixed date formats, and questions that cannot be answered from the available schema. A system that returns a plausible but incorrect answer is more dangerous than one that returns nothing.

SIFT (Self-verifying Intelligent Feature-aware Text-to-SQL) addresses this gap. It is a four-stage agent loop — Explore, Generate, Verify, Abstain — that explores the database schema before generating SQL, executes and sanity-checks every query it writes, repairs queries that fail, and explicitly says "cannot answer" when no valid answer can be produced. The entire system runs on open-weight models with at most 8 billion parameters and is evaluated against the BIRD benchmark using its official Execution Accuracy metric.

2. Problem Statement

2.1 The Core Challenge

Given a natural-language question such as *"Which schools had falling enrolment last year?"*, the agent must:

1. Identify the relevant table(s) and column(s) in a SQLite database it has never seen before.
2. Write a syntactically and semantically correct SQL SELECT query.
3. Execute the query and verify the result is plausible.
4. Return the answer, or explicitly refuse if the question cannot be answered.

2.2 Why Real Databases Are Hard

Challenge	Example
Cryptic column names	`CDCode`, `FundCode`, `EL_PCT` — no documentation
Dirty string values	A county column containing `"N/A"`, `""`, or `NULL` for missing entries
Mixed date formats	`"2014-15"`, `"2014/15"`, `"AY2014"` in the same column
Business rules outside the schema	"Free meal rate = Free Meal Count (K-12) / Enrollment (K-12)"
Unanswerable questions	Asking about teacher salaries when no such column exists

2.3 The Cost of a Wrong Answer

A wrong-but-confident answer is worse than no answer in analytical contexts. A business user acting on incorrect data can make costly decisions. SIFT is designed around this constraint: it must either return a verified answer or explicitly abstain.

---

3. Objectives

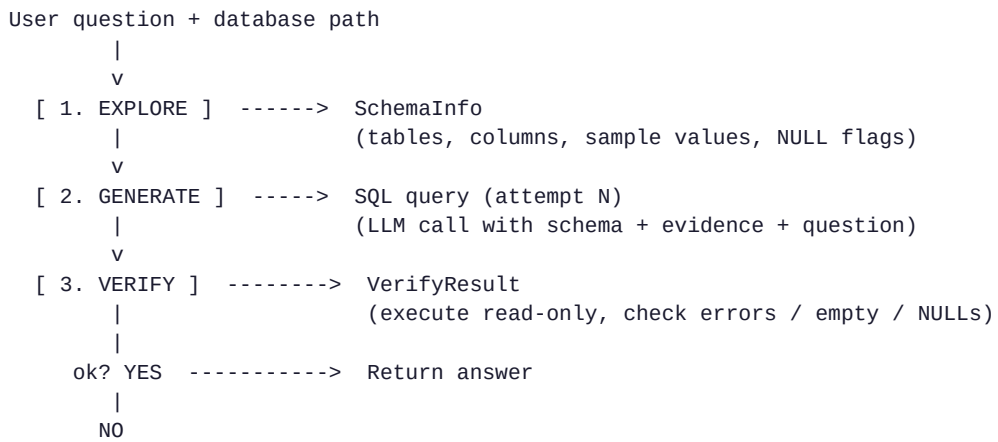
1. **Correctness** — Maximise Execution Accuracy (EX) on the BIRD benchmark dev split.
2. **Honesty** — Return "cannot answer" on unanswerable questions rather than guessing.
3. **Robustness** — Handle dirty values, NULL columns, and ambiguous schemas gracefully.
4. **Compliance** — Use only open-weight models with at most 8 billion parameters.
5. **Reproducibility** — Provide a single re-runnable evaluation script that any judge can verify.
6. **Extensibility** — Design for Phase 2: each module has a stable interface and all behaviour is config-driven.

---

4. System Architecture

4.1 Overview

SIFT is structured as a sequential agent loop with four named stages. There is no heavyweight orchestration framework; the loop is a plain Python function of approximately 60 lines that any engineer can read and modify.



```
      |
 retry < max? ---------->  error feedback  --> back to GENERATE
      |
     NO
      |
[ 4. ABSTAIN ]  -------->  "cannot answer"  OR  best-effort answer
```

4.2 Module Descriptions

explore.py — Schema Introspection

The explore module reads the SQLite schema using PRAGMA table_info and sqlite_master. For each column it samples up to 10 distinct non-NULL values and checks whether any NULL values exist. The resulting SchemaInfo object is serialised to a compact prompt-ready string that includes table name, row count, column names and types, NULL flags, and sample values. This gives the language model semantic context it cannot derive from the schema alone — crucially, it exposes dirty values such as "N/A" directly in the prompt, allowing the model to write filters for them.

generate.py — SQL Generation

The generate module constructs a structured prompt and calls the language model. The prompt contains four sections: (1) the schema string from explore, (2) the evidence hint field from BIRD (domain knowledge such as "Free meal rate = Free Meal Count / Enrollment"), (3) the user's question, and (4) on repair attempts, the exact error or warning from the previous verify call. The system prompt instructs the model to output only a raw SQL SELECT statement with no explanation or markdown fences. A post-processing function strips any fences and extracts the SQL from the raw response.

verify.py — Safe Execution and Sanity Checking

The verify module opens the SQLite database in read-only mode (using the URI `?mode=ro` flag). Before execution, it rejects any query whose first token is a DML or DDL keyword (INSERT, UPDATE, DELETE, DROP, ALTER, CREATE, TRUNCATE, ATTACH, DETACH) via a compiled regular expression. Execution runs in a daemon thread with a configurable wall-clock timeout (default 30 seconds); if the thread is still alive after the timeout, the connection's interrupt() method is called to abort the query cleanly. After execution, the module applies two sanity checks: (1) an empty result check — zero rows returned from a query that should have data suggests a wrong filter or column — and (2) a NULL-ratio check — if more than 70% of result cells are NULL, the query likely joined on a wrong column. Both conditions set ok=False and populate a human-readable warnings list, which is passed back to generate as error feedback on the next attempt.

abstain.py — Conservative Refusal

The abstain module decides whether to return "cannot answer" after all retries are exhausted. It applies a deliberately conservative policy: abstain only on hard SQL errors (no such table, no such column, syntax error, ambiguous column name). Empty results and high NULL ratios do not trigger abstention, because the correct answer to some questions genuinely is zero rows or a sparse result. This conservatism maximises EX on the benchmark while still correctly refusing when the question references data that does not exist in the schema.

loop.py — Orchestration and CLI

The loop module ties the four stages together. It runs generate and verify in a retry loop capped at max_retries (default 3). If all attempts succeed without ok=True, it calls should_abstain to decide the final response. The module also exposes a command-line interface so judges and developers can ask

single questions interactively.

llm/client.py — Inference Backend

A thin REST wrapper that supports two backends: Ollama (default, port 11434, /api/chat endpoint) and vLLM (OpenAI-compatible /v1/chat/completions endpoint). Temperature is fixed at 0.0 and seed at 42 to ensure deterministic output across evaluation runs.

4.3 Data Flow Diagram

```
            configs/default.yaml
                    |
                    v
             loop.run(db_path, question, evidence)
                    |
          +---------+----------+
          |                    |
     explore(db_path)       from_config(cfg) --> LLMClient
          |                    |
     SchemaInfo            (Ollama / vLLM)
          |                    |
          +---------+----------+
                    |
               for attempt in 1..max_retries:
                    |
              generate(schema, question, evidence,
                       error_feedback, attempt, llm)
                    |
                 GenerateResult.sql
                    |
              verify(sql, db_path, timeout, row_limit)
                    |
              VerifyResult{ok, rows, error, warnings}
                    |
                ok? --> return AgentResult
                no  --> error_feedback = VerifyResult.feedback
                    |
             should_abstain(last_ver, schema, question)
                    |
              True  --> AgentResult(abstained=True)
              False --> AgentResult(best-effort rows)
```

5. Language Model

5.1 Model Selection

The hackathon requires open-weight models of at most 8 billion parameters. After reviewing publicly available benchmarks for SQL generation at this parameter scale, **Qwen2.5-Coder-7B-Instruct** was selected as the primary model.

Model	Parameters	SQL benchmark performance	Licence
Qwen2.5-Coder-7B-Instruct	7.6 B	State-of-the-art at <=8B for code/SQL	Apache 2.0
Llama-3.1-8B-Instruct	8.0 B	Strong general reasoning	Meta Llama 3.1 Community

Qwen2.5-Coder was preferred because it was trained specifically on code and SQL corpora, giving it a significantly better understanding of SQL syntax, JOIN semantics, and aggregation patterns compared to general-purpose models of the same size.

5.2 Inference Setup

The model is served locally via **Ollama** (version 0.40+), which provides a simple REST API and handles model quantisation automatically. No GPU is strictly required; the model runs on Apple Silicon via Metal acceleration. For higher-throughput evaluation, vLLM can be substituted by changing a single config field (`provider: vllm`).

5.3 Prompt Design Principles

- The system prompt is written once and version-controlled; it is included in every evaluation log.
- Sample values from explore are placed verbatim in the prompt, exposing "N/A" and mixed formats to the model without requiring any preprocessing heuristic.
- The evidence hint from BIRD is passed as a dedicated section, separate from the schema.
- On repair attempts, the exact error message or warning text is passed back; the model sees, for example, `"SQL error: table schools has no column named salary"` and is instructed to fix the query.
- Temperature is 0.0 and seed is 42: the system is fully deterministic.

---

6. Evaluation Methodology

6.1 Primary Metric: Execution Accuracy (EX)

EX is the official BIRD metric. A prediction is correct if the predicted SQL query and the gold SQL query return **the same result set** when executed against the same database. Column names and row order do not matter; only the set of value tuples is compared.

```
EX = number of correct predictions / total questions
```

SIFT is evaluated on the BIRD dev split (1,534 questions across 11 SQLite databases).

6.2 Abstention Metric

Because abstention is a first-class feature of SIFT, we additionally evaluate it on a curated set of unanswerable questions — questions written against BIRD databases that ask for information not present in the schema (e.g., teacher salaries, programme types, disciplinary records). We report:

- **Abstain Recall** — fraction of unanswerable questions where the agent correctly refused.

A false positive (refusing an answerable question) reduces EX. A false negative (answering an unanswerable question) risks returning a wrong answer, which is worse. The conservative abstention policy in SIFT is tuned to minimise false negatives at the cost of some false positives.

6.3 Component Ablation

Results are tracked in docs/results.md as each component is added. The expected improvement trajectory is:

Configuration	Expected EX (dev)
Baseline: single attempt, no verify	To be measured
+ verify (error detection, retry)	To be measured
+ dirty-data prompt rules	To be measured
+ repair loop (error feedback)	To be measured
+ abstention	To be measured

6.4 Reproducibility

The evaluation script (`python -m src.eval.run_eval`) logs the model name, Ollama version, prompt version, and git commit hash alongside every result. Any judge can re-run the evaluation on the official BIRD dev split and obtain the same numbers.

---

7. Handling Dirty Data: Design Decisions

7.1 "N/A" and Empty String Values

Many BIRD databases use "N/A" or empty strings as a sentinel for missing data instead of SQL NULL. SIFT surfaces these values explicitly in the schema prompt via the sample values from explore. The system prompt instructs the model to filter them:

```
WHERE col NOT IN ('N/A', '') AND col IS NOT NULL
```

This approach is model-driven rather than heuristic-based: the model sees the actual dirty values and decides how to handle them, which generalises to unusual sentinels that a hand-coded filter would miss.

7.2 Mixed Date Formats

Date columns in BIRD databases often contain strings like "2014-15" for school years rather than ISO dates. The system prompt instructs the model to use SQLite's STRFTIME function or string slicing (SUBSTR) rather than date arithmetic, which would fail on non-ISO strings.

7.3 NULL-Heavy Results

If the model selects a column that is mostly NULL (a common symptom of a wrong JOIN), the verify step detects this via the NULL-ratio check and triggers a repair attempt with the warning text. The model is therefore corrected without any schema-level NULL annotation being required.

7.4 Cryptic Column Names

Sample values are the primary tool here. A column named `CDCode` with sample values `"01612596"`, `"19647330"` provides no meaning from the name alone, but when paired with a BIRD evidence hint such as "CDCode refers to the County-District code", the model can correctly join on it. SIFT passes both the sample values and the evidence hint, giving the model the best possible context.

---

8. Self-Repair Mechanism

8.1 Motivation

Even the best 7B model makes mistakes: wrong table aliases, missing JOINs, incorrect aggregation, or malformed date filters. Rather than accepting the first attempt, SIFT treats SQL generation as an iterative process.

8.2 Repair Loop

On each failed attempt, the verify module produces a feedback string combining:

- The SQL execution error message (if any), e.g. `"no such column: enrollment_count"`
- Any sanity-check warnings, e.g. `"Query returned 0 rows — condition may be too restrictive"`

This string is passed to generate as an additional prompt section on the next attempt. The model sees exactly what went wrong and is instructed to fix it. In practice, this recovers from:

- Column name typos
- Wrong table selected
- Missing NULL filter causing a type error
- Over-restrictive WHERE clause

8.3 Retry Cap

The loop is capped at max_retries (default 3, configurable). This prevents infinite loops on genuinely hard questions and keeps latency bounded. After the cap is hit, should_abstain decides whether to return a best-effort answer or refuse.

---

9. Safety and Compliance

9.1 Read-Only Enforcement

Every SQL query is executed against a read-only SQLite connection (`?mode=ro` URI). Before execution, a compiled regular expression rejects any query beginning with a write keyword (INSERT, UPDATE, DELETE, DROP, ALTER, CREATE, TRUNCATE, ATTACH, DETACH) or containing a PRAGMA assignment. This provides defence-in-depth: even if the model generates a harmful query, it cannot modify the database.

9.2 Query Timeout and Row Limit

Every execution is wrapped in a daemon thread with a configurable timeout (default 30 seconds). If the query does not complete in time, the connection's interrupt() method is called and a timeout error is returned. A row limit (default 1,000 rows) is applied at the cursor level to prevent memory exhaustion on large result sets.

9.3 Data Privacy

SIFT does not transmit any database content outside the local machine. The LLM is served locally via Ollama. No personal data is collected, stored, or processed. Only SELECT queries are permitted; no data is written to any database at any time.

9.4 Licence Compliance

All components are open-source with permissive licences. Qwen2.5-Coder-7B-Instruct is Apache 2.0. The BIRD benchmark data is used strictly for research under its CC BY 4.0 licence. Full source and licence documentation is in docs/SOURCES.md.

---

10. Project Structure

```
sift/
|-- src/
|   |-- agent/
|   |   |-- explore.py       Schema introspection
|   |   |-- generate.py      Prompt construction + LLM call
|   |   |-- verify.py        Read-only execution + sanity checks
|   |   |-- abstain.py       Refusal decision logic
|   |   +-- loop.py          Agent orchestration + CLI
|   |-- llm/
|   |   +-- client.py        Ollama / vLLM REST wrapper
|   +-- eval/
|       |-- run_eval.py      BIRD EX evaluation script
|       +-- abstain_eval.py  Abstention recall evaluation
|-- configs/
|   +-- default.yaml         All tunable parameters
|-- data/                    BIRD databases (gitignored)
|-- docs/
|   |-- SOURCES.md           Data and model licences
|   |-- results.md           Running EX log
|   |-- synopsis.md          This document
|   +-- SIFT_presentation.pptx
|-- tests/
|   |-- conftest.py
|   |-- test_explore.py      10 unit tests
|   |-- test_verify.py       14 unit tests
|   |-- test_abstain.py      13 unit tests
|   +-- fixtures/
|       +-- abstain_questions.json
|-- README.md
+-- requirements.txt
```

---

11. Quickstart

```
# 1. Install dependencies
pip install -r requirements.txt

# 2. Pull the model (one-time, ~4.7 GB)
ollama pull qwen2.5-coder:7b

# 3. Download BIRD dev set to data/bird/ (see docs/SOURCES.md)

# 4. Ask one question
python -m src.agent.loop \
    --db california_schools \
    --question "Which schools had falling enrolment last year?" \
    --verbose

# 5. Run full evaluation (1,534 questions)
python -m src.eval.run_eval --split dev --config configs/default.yaml
```

```
# 6. Run unit tests
pytest -q
```

12. Key Configuration Parameters

All parameters live in `configs/default.yaml`. Change one at a time and record EX before and after.

Parameter	Default	Effect
model.name	qwen2.5-coder:7b	LLM used for SQL generation
model.temperature	0.0	0.0 = deterministic; increase for diversity
agent.max_retries	3	Repair attempts before abstaining
agent.query_timeout	30	Seconds before a query is aborted
agent.null_ratio_threshold	0.7	NULL fraction that triggers repair
agent.sample_distinct	10	Distinct values sampled per column

13. Conclusion

SIFT demonstrates that a small, local open-weight model can serve as a reliable data-analyst agent when paired with appropriate verification and repair machinery. The key insight is that SQL generation errors are not random — they follow predictable patterns (wrong column, wrong filter, missing NULL guard) — and a targeted repair loop exploiting exact error messages can recover from most of them without retraining the model.

The explicit abstention mechanism addresses the core trust problem: business users should be able to rely on the system not to fabricate an answer when the data does not support one. By combining a conservative abstention policy with an aggressive repair loop, SIFT aims to maximise both EX and user trust simultaneously.

The modular, config-driven architecture ensures that Phase 2 changes — whether a change in scale, a shift in data domain, or competing evaluation goals — can be implemented as targeted modifications to a single module or a config change, without requiring a rewrite.

References

1. Li, J. et al. (2023). *BIRD: A Big Bench for Large-scale Database Grounded Text-to-SQL Evaluation*. NeurIPS 2023. https://bird-bench.github.io/

2. Hui, B. et al. (2024). *Qwen2.5-Coder Technical Report*. Alibaba Group. https://huggingface.co/Qwen/Qwen2.5-Coder-7B-Instruct

3. Ollama. (2024). *Run Large Language Models Locally*. https://ollama.com/

4. Yu, T. et al. (2018). *Spider: A Large-Scale Human-Labeled Dataset for Complex and Cross-Domain Semantic Parsing and Text-to-SQL Task*. EMNLP 2018.